

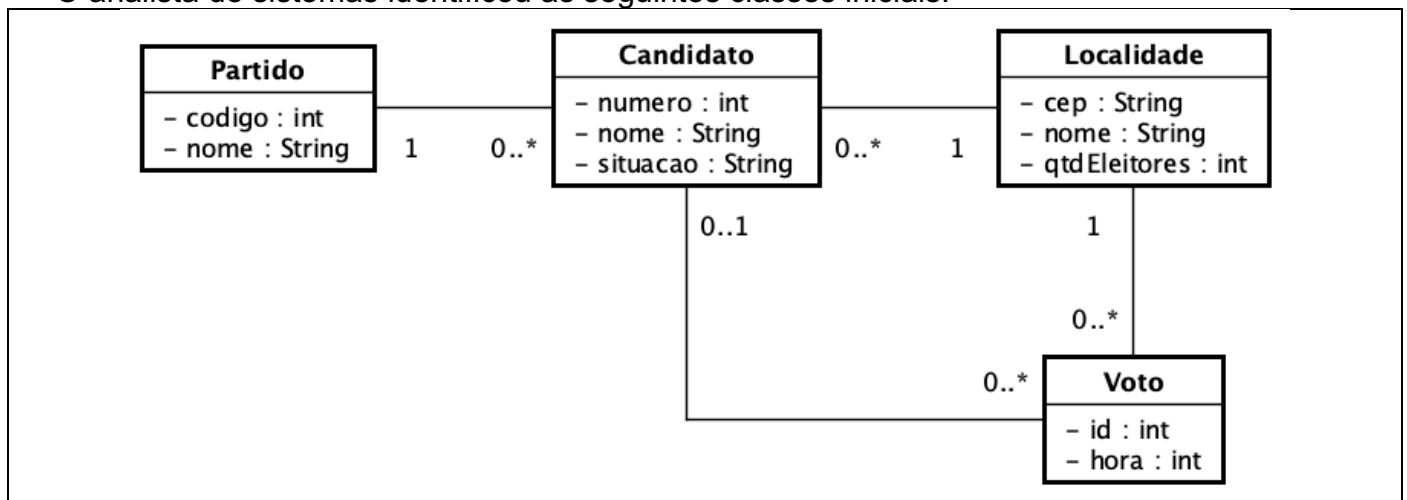
Trabalho I

Eleições

1. Enunciado geral:

A ACMEPolling é uma empresa independente que monitora a votação de candidatos e que deseja automatizar seu negócio.

O analista de sistemas identificou as seguintes classes iniciais:



Todo candidato inicialmente é cadastrado como *pré-candidato*. Após análise externa torna-se *elegível* ou *inelegível* ou *removido*. Se estiver *inelegível*, pode se tornar *elegível* novamente ou *removido* antes do período da eleição. Apenas se for *elegível* pode receber votos. Durante, ou ao final da votação, pode-se tornar *inelegível*. Ao final da apuração torna-se *eleito* ou *não eleito*.

A votação ocorre em um único dia dentro de um período de horas (entre 8:00 e 17:00).

Um voto é invalidado se estiver fora do período de horas da votação, se o número do candidato for inválido, se o candidato votado é de outra localidade do voto, se a quantidade de votos é maior que a quantidade de eleitores da localidade.

O candidato eleito de uma localidade é aquele *elegível* que obtiver a maioria dos votos válidos. Em caso de empate, vence quem teve o último voto mais cedo.

As funcionalidades básicas do sistema devem ser:

1. Consultar todos os candidatos cadastrados (com respectivos partido e localidade).
2. Consultar todos os candidatos de uma determinada localidade e situação.
3. Cadastrar um novo candidato.
4. Cadastrar um novo voto.
5. Consultar o candidato eleito de uma determinada localidade.
6. Consultar a quantidade de votos de um determinado candidato.
7. Consultar a votação dos candidatos de uma determinada localidade.
8. Consultar os candidatos mais votados de cada partido.
9. Alterar a situação de um candidato.
10. Remover um candidato (apenas logicamente).

2. Definição do trabalho:

Nesta seção são detalhados os *endpoints* correspondentes às funcionalidades esperadas. Para cada *endpoint* serão detalhados a rota, os parâmetros recebidos e o JSON resposta.

Por uma questão de dimensionamento de escopo, serão omitidos deste trabalho os *endpoints* relativos ao cadastramento/edição todas as entidades. Para que o sistema possa ser testado a contento, deve ser previsto um trecho de código de inicialização capaz de inserir dados no sistema com pelo menos 3 partidos distintos e 3 localidades diferentes. Embora o sistema deva incluir os *endpoints* relativos ao gerenciamento de candidatos, o mesmo trecho de código de inicialização do sistema deverá prever pelo menos 3 candidatos elegíveis (1 para cada partido cadastrado, sendo 2 da mesma localidade).

1) *Endpoint*: GET /acmepolling/cadastro/listacandidatos

Descrição	Lista com todos os candidatos cadastrados (com respectivos partido e localidade)
Parâmetros de entrada	Nenhum
JSON resposta	[{numero, nome, situação, nome_partido, nome_localidade}, ...]

2) *Endpoint*: GET /acmepolling/cadastro/listacandidatoslocalidade/:cep/situação/:situacao

Descrição	Lista com todos os candidatos de uma localidade em uma determinada situação
Parâmetros de entrada	:cep – CEP da localidade :situação – situação do candidato (PRECANDIDATO, ELEGIVEL, INELEGIVEL, ELEITO, NAOELEITO ou REMOVIDO)
JSON resposta	[{numero, nome} , ...]

3) *Endpoint*: POST /acmepolling/cadastro/cadcandidato

Descrição	Cadastra um novo candidato
Corpo da requisição	{numero,nome,codigo,cep}
JSON resposta	Retorna se o cadastro teve sucesso Booleano: true ou false

4) *Endpoint*: POST /acmepolling/votacao/cadvoto

Descrição	Cadastra um novo voto em candidato
Corpo da requisição	{id,hora,numero,cep}
JSON resposta	Retorna se o cadastro teve sucesso Booleano: true ou false

5) *Endpoint*: GET /acmepolling/apuracao/consultaeleito

Descrição	Consulta o candidato eleito de uma determinada localidade
Parâmetros de entrada	cep

JSON resposta	{numero,nome,nome_partido,quantidade total de votos}
------------------	--

6) *Endpoint:* GET /acmepolling/apuracao/consultacandidato

Descrição	Consulta a quantidade de votos de um determinado candidato
Parâmetros de entrada	numero
JSON resposta	{quantidade de votos válidos,quantidade de votos inválidos }

7) *Endpoint:* GET /acmepolling/apuracao/consultalocalidade

Descrição	Consulta a votação dos candidatos de um determinada localidade
Parâmetros de entrada	cep
JSON resposta	[[numero, nome, quantidade de votos válidos,quantidade de votos inválidos], ...]

8) *Endpoint:* GET /acmepolling/apuracao/listamaisvotadospartido

Descrição	Lista os candidatos mais votados de cada partido
Parâmetros de entrada	Nenhum
JSON resposta	[[nome_partido,numero,nome,quantidade de votos válidos], ...]

9) *Endpoint:* PUT /acmepolling/cadastro/atualizacandidato/:numero/situacao/:status

Descrição	Atualiza a situação de um candidato
Parâmetros de entrada	:numero - número do candidato :status - nova situação do candidato
JSON resposta	Retorna o cadastro completo do candidato {numero, nome, situação, nome_partido, nome_localidade}

10) *Endpoint:* DELETE /acmepolling/cadastro/removecandidato

Descrição	Remove um candidato (apenas logicamente, não fisicamente)
Corpo da requisição	numero
JSON resposta	Booleano: true (se removeu) ou false (se não removeu)

3. Critérios de avaliação

O trabalho será avaliado considerando os conceitos abordados na disciplina e conforme os seguintes critérios:

- Desenvolvimento das entidades (classes de dados): 1 ponto.
- Desenvolvimento e execução dos *endpoints*: 3 pontos.
- Desenvolvimento e execução de testes de *endpoints*: 3 pontos.

- Relatório: 3 pontos.
 - Diagrama de classes UML atualizado.
 - Diagrama de estados UML de um candidato.
 - Indicar quais e onde foram utilizados padrões de projeto e de arquitetura.
- O *backend* deverá ser desenvolvido no *framework* Spring-Boot.
- Compartilhamento de acesso ao repositório privado do GitHub do exercício com o professor (mhyamaguti).
- A apresentação é obrigatória:
 - A avaliação da execução será na apresentação.
 - Em caso de ausência na apresentação, a nota será, no máximo, de 50% do total.

4. Entrega e apresentação:

- Na criação do repositório no GitHub nomear: FDS-Trab1-NomeSobrenome. Ex.: FDS-Trab1-MariaSilvaPedroSouza
- Entrega dos códigos-fonte *scripts* de teste e diagramas (compactados em um único arquivo .zip ou .rar) no Moodle.
- Data de entrega e apresentação: **8 / 10 / 2026**.

5. Considerações finais:

- O trabalho pode ser desenvolvido individualmente ou em dupla.
 - Em caso de dupla, na apresentação um computador será o servidor e outro computador será o cliente.
- Não serão aceitos trabalhos com erros de compilação. Programas que não compilarem corretamente terão nota zerada.
- A cópia parcial ou completa do trabalho terá como consequência a atribuição de nota ZERO ao trabalho dos alunos envolvidos.